Gandharav Finchartg Enterprises LLP - Full Stack CRM Website Code

1. BACKEND SETUP

backend/package.json

```
{
  "name": "crm-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.5.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

---

backend/.env

```
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=gandharav_secret_key
```

---

backend/server.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const dotenv = require('dotenv');

const authRoutes = require('./routes/authRoutes');
const clientRoutes = require('./routes/clientRoutes');

dotenv.config();

const app = express();

app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URI)
.then(() => console.log('MongoDB Connected'))
.catch((err) => console.log(err));

app.use('/api/auth', authRoutes);
app.use('/api/clients', clientRoutes);

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server Running on ${PORT}`);
});
```

---

backend/models/User.js

```
const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  branch: {
    type: String,
    required: true
  },
```

```
  location: {
    type: String,
    required: true
  },
  mobile: {
    type: String,
    required: true
  },
  password: {
    type: String,
    required: true
  },
  dob: {
    type: String,
    required: true
  },
  post: {
    type: String,
    required: true
  },
  rank: {
    type: String,
    required: true
  },
  userId: {
    type: String,
    unique: true
  }
});

module.exports = mongoose.model('User', userSchema);
```

---

backend/models/Client.js

```
const mongoose = require('mongoose');

const clientSchema = new mongoose.Schema({
  clientName: String,
  location: String,
  mobile: String,

  accountType: String,

  companyAccountNo: String,
```

```
  companyIFSC: String,
  companyBankLocation: String,

  clientAccountNo: String,
  clientIFSC: String,
  clientBankLocation: String,

  amountDate: String,
  amount: Number,
  totalAmount: Number,
  utrNo: String,

  companyName: String
}, {
  timestamps: true
});

module.exports = mongoose.model('Client', clientSchema);
```

---

backend/middleware/authMiddleware.js

```
const jwt = require('jsonwebtoken');

module.exports = function(req, res, next) {

  const token = req.header('Authorization');

  if(!token) {
    return res.status(401).json({ message: 'No Token' });
  }

  try {
    const verified = jwt.verify(token, process.env.JWT_SECRET);
    req.user = verified;
    next();
  } catch(err) {
    res.status(401).json({ message: 'Invalid Token' });
  }
}
```

---

backend/routes/authRoutes.js

```
const router = require('express').Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

const User = require('../models/User');

router.post('/register', async(req, res) => {

  try {

    const {
      name,
      branch,
      location,
      mobile,
      password,
      dob,
      post,
      rank
    } = req.body;

    const hashedPassword = await bcrypt.hash(password, 10);

    const lastName = name.split(' ').slice(-1)[0].toUpperCase();

    const randomNo = Math.floor(100 + Math.random() * 900);

    const userId = `${branch.toUpperCase()}-${lastName}-${randomNo}`;

    const newUser = new User({
      name,
      branch,
      location,
      mobile,
      password: hashedPassword,
      dob,
      post,
      rank,
      userId
    });

    await newUser.save();

    res.json({
```

```
      success: true,
      message: 'User Registered Successfully',
      userId
    });

  } catch(err) {
    res.status(500).json(err);
  }
});

router.post('/login', async(req, res) => {

  try {

    const { userId, password } = req.body;

    const user = await User.findOne({ userId });

    if(!user) {
      return res.status(400).json({ message: 'User Not Found' });
    }

    const isMatch = await bcrypt.compare(password, user.password);

    if(!isMatch) {
      return res.status(400).json({ message: 'Wrong Password' });
    }

    const token = jwt.sign(
      { id: user._id },
      process.env.JWT_SECRET,
      { expiresIn: '1d' }
    );

    res.json({
      success: true,
      token,
      user
    });

  } catch(err) {
    res.status(500).json(err);
  }
});

module.exports = router;
```

backend/routes/clientRoutes.js

```
const router = require('express').Router();
const Client = require('../models/Client');
const authMiddleware = require('../middleware/authMiddleware');

router.post('/add', authMiddleware, async(req, res) => {

  try {

    const client = new Client(req.body);

    await client.save();

    res.json({
      success: true,
      message: 'Client Added Successfully'
    });

  } catch(err) {
    res.status(500).json(err);
  }
});

router.get('/', authMiddleware, async(req, res) => {

  try {

    const clients = await Client.find();

    res.json(clients);

  } catch(err) {
    res.status(500).json(err);
  }
});

router.put('/:id', authMiddleware, async(req, res) => {

  try {

    await Client.findByIdAndUpdate(req.params.id, req.body);

    res.json({
      success: true,
      message: 'Client Updated Successfully'
```

```
    });

  } catch(err) {
    res.status(500).json(err);
  }
});

router.delete('/:id', authMiddleware, async(req, res) => {

  try {

    await Client.findByIdAndDelete(req.params.id);

    res.json({
      success: true,
      message: 'Client Deleted Successfully'
    });

  } catch(err) {
    res.status(500).json(err);
  }
});

module.exports = router;
```

---

2. FRONTEND SETUP

frontend/package.json

```
{
  "name": "crm-frontend",
  "version": "1.0.0",
  "private": true,
  "dependencies": {
    "axios": "^1.7.2",
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "react-router-dom": "^6.24.1",
    "react-csv": "^2.2.2"
  },
  "scripts": {
    "start": "react-scripts start"
```

```
  }
}
```

---

frontend/src/App.js

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';

function App() {

  return (
    <BrowserRouter>
      <Routes>
        <Route path='/' element={<Login />} />
        <Route path='/register' element={<Register />} />
        <Route path='/dashboard' element={<Dashboard />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App;
```

---

frontend/src/pages/Register.js

```
import React, { useState } from 'react';
import axios from 'axios';

function Register() {

  const [formData, setFormData] = useState({
    name: '',
    branch: '',
    location: '',
    mobile: '',
    password: '',
    dob: '',
    post: '',
```

```
    rank: ''
  });

  const handleChange = (e) => {
    setFormData({
      ...formData,
      [e.target.name]: e.target.value
    });
  }

  const handleSubmit = async() => {

    try {

      const res = await axios.post(
        'http://localhost:5000/api/auth/register',
        formData
      );

      alert(`Registration Successful. ID: ${res.data.userId}`);

    } catch(err) {
      alert('Registration Failed');
    }
  }

  return (
    <div>

      <h1>Register Director / Co-Director</h1>

      <input name='name' placeholder='Name' onChange={handleChange} />
      <input name='branch' placeholder='Branch' onChange={handleChange} />
      <input name='location' placeholder='Location' onChange={handleChange} />
      <input name='mobile' placeholder='Mobile No' onChange={handleChange} />
      <input name='password' type='password' placeholder='Password'
onChange={handleChange} />
      <input name='dob' type='date' onChange={handleChange} />
      <input name='post' placeholder='Post' onChange={handleChange} />
      <input name='rank' placeholder='Rank' onChange={handleChange} />

      <button onClick={handleSubmit}>Register</button>

    </div>
  )
}

export default Register;
```

frontend/src/pages/Login.js

```
import React, { useState } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';

function Login() {

  const [userId, setUserId] = useState('');
  const [password, setPassword] = useState('');

  const navigate = useNavigate();

  const handleLogin = async() => {

    try {

      const res = await axios.post(
        'http://localhost:5000/api/auth/login',
        {
          userId,
          password
        }
      );

      localStorage.setItem('token', res.data.token);

      navigate('/dashboard');

    } catch(err) {
      alert('Login Failed');
    }
  }

  return (
    <div>

      <h1>Login</h1>

      <input
        placeholder='ID'
        onChange={(e) => setUserId(e.target.value)}
      />

      <input
```

```
        type='password'
        placeholder='Password'
        onChange={(e) => setPassword(e.target.value)}
      />

      <button onClick={handleLogin}>Login</button>

    </div>
  )
}

export default Login;
```

---

frontend/src/pages/Dashboard.js

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';
import { CSVLink } from 'react-csv';

function Dashboard() {

  const [clients, setClients] = useState([]);

  const [formData, setFormData] = useState({
    clientName: '',
    location: '',
    mobile: '',
    accountType: '',

    companyAccountNo: '',
    companyIFSC: '',
    companyBankLocation: '',

    clientAccountNo: '',
    clientIFSC: '',
    clientBankLocation: '',

    amountDate: '',
    amount: '',
    totalAmount: '',
    utrNo: '',

    companyName: 'Gandharav Finchartg Enterprises LLP'
  });
```

```
const token = localStorage.getItem('token');

const headers = {
  Authorization: token
}

const fetchClients = async() => {

  const res = await axios.get(
    'http://localhost:5000/api/clients',
    { headers }
  );

  setClients(res.data);
}

useEffect(() => {
  fetchClients();
}, []);

const handleChange = (e) => {

  setFormData({
    ...formData,
    [e.target.name]: e.target.value
  });
}

const addClient = async() => {

  await axios.post(
    'http://localhost:5000/api/clients/add',
    formData,
    { headers }
  );

  alert('Client Added');

  fetchClients();
}

return (
  <div>

    <h1>Dashboard</h1>

    <h2>Add Client</h2>
```

```
      <input name='clientName' placeholder='Client Name'
onChange={handleChange} />
      <input name='location' placeholder='Location' onChange={handleChange} />
      <input name='mobile' placeholder='Mobile' onChange={handleChange} />
      <input name='accountType' placeholder='Account Type'
onChange={handleChange} />

      <input name='companyAccountNo' placeholder='Company Account Number'
onChange={handleChange} />
      <input name='companyIFSC' placeholder='Company IFSC'
onChange={handleChange} />
      <input name='companyBankLocation' placeholder='Company Bank Branch'
onChange={handleChange} />

      <input name='clientAccountNo' placeholder='Client Account Number'
onChange={handleChange} />
      <input name='clientIFSC' placeholder='Client IFSC'
onChange={handleChange} />
      <input name='clientBankLocation' placeholder='Client Bank Branch'
onChange={handleChange} />

      <input name='amountDate' type='date' onChange={handleChange} />
      <input name='amount' placeholder='Amount' onChange={handleChange} />
      <input name='totalAmount' placeholder='Total Amount'
onChange={handleChange} />
      <input name='utrNo' placeholder='UTR Number' onChange={handleChange} />

      <button onClick={addClient}>Add Client</button>

      <hr />

      <CSVLink data={clients} filename='clients.csv'>
        Download Clients
      </CSVLink>

      <h2>Client List</h2>

      {
        clients.map((client) => (
          <div key={client._id}>
            <h3>{client.clientName}</h3>
            <p>{client.mobile}</p>
            <p>{client.amount}</p>
            <p>{client.utrNo}</p>
          </div>
        ))
      }
```

```
    </div>
  )
}

export default Dashboard;
```

---

3. INSTALL COMMANDS

Backend

```
cd backend
npm install
npm run dev
```

Frontend

```
cd frontend
npm install
npm start
```

---

4. MONGODB DATABASE

Create MongoDB Atlas account:

https://www.mongodb.com/cloud/atlas

Paste connection string into:

```
MONGO_URI=your_connection_string
```

---

5. FINAL FEATURES INCLUDED

✅ Director Registration ✅ Co-Director Registration ✅ Auto Generated ID ✅ Login System ✅ JWT Authentication ✅ Add Client ✅ Edit Client API ✅ Delete Client API ✅ View Clients ✅ Download Client CSV ✅ MongoDB Database ✅ React Frontend ✅ Node Backend ✅ Secure Password Hashing

---

6. DEPLOYMENT

Frontend

Deploy on:

- Vercel
- Netlify

Backend

Deploy on:

- Render
- Railway

Database

Use:

- MongoDB Atlas